**Technik der MMI**, UXD-B
Praktikum, WiSe 2026/27

**Praktikumsblatt 5**
Prof. Dr. Andreas Riener, Prof. Dr. Simon Nestler

# Arduino und vvvv koppeln

### Worum geht es in diesem Blatt?

Sie koppeln die beiden Werkzeuge der vorigen Blätter: Der Arduino liefert Messwerte, vvvv gamma verarbeitet sie und zeigt sie an. Zuerst richten Sie die Verbindung über die serielle Schnittstelle von Hand ein, dann wechseln Sie auf das Firmata-Protokoll, das die Kopplung deutlich vereinfacht. Am Ende haben Sie eine Anwendung, die eine Temperatur über eine RGB-LED sichtbar macht.

### Was erwarten wir von der Dokumentation (Protokoll)?

- Beantworten Sie die gestellten Fragen in einem Protokoll (Moodle-Fragebogen/Quiz)
- Machen Sie zusätzlich Screenshots Ihrer Patches/Programme und Fotos, die Ihre Lösungen in „Aktion“ zeigen (vvvv-Screenshots erstellen Sie direkt in vvvv mit der Tastenkombination „**STRG + 2**“); vergessen Sie nicht, die Patches zu speichern!
- Halten Sie Ihre **eigene Interpretation** fest: Was haben Sie im Experiment erkannt, was ist Ihnen aufgefallen?
- Welche **Probleme** sind Ihnen begegnet, und wie haben Sie sie **gelöst**?

### Wie?

- **Bearbeiten Sie zur Abgabe den Test „Abgabe Praktikumsblatt 5“** in Moodle (Kursraum Technik der Mensch-Maschine Interaktion).
- **Kopieren Sie dazu Ihre Antworten, Screenshots, Fotos usw. in die jeweiligen Antwortfelder.** Das Protokoll können Sie in der Praktikumsstunde direkt im Angabe-Worddokument oder in einem separaten Textdokument schreiben.
- **Speichern Sie die vvvv-Patches und geben Sie sie als Anhänge mit ab!**
- In Zweiergruppen lädt nur die protokollführende Person hoch; im Protokoll stehen beide mit vollständigem Namen.
- **ACHTUNG: Chromium-basierte Browser können offenbar keine Bilder kopieren. Nutzen Sie in dem Fall einen anderen Browser, z. B. Firefox.**

### Abgabe?

- **Abgabe innerhalb von 48 Stunden** nach Ihrem Praktikumstermin, in Moodle

Hier kombinieren Sie die Vielseitigkeit eines Arduino-Boards (Blatt 3) mit dem Komfort und der Mächtigkeit von „*vvvv*“ (Blatt 2). So entwickeln Sie in kürzester Zeit Prototypen und Systeme, die mit beinahe beliebigen Ein- und Ausgabegeräten (Sensoren und Aktuatoren) funktionieren. Das eröffnet nahezu „unendliche“ Möglichkeiten für eigene kreative Projekte.

**Nutzen Sie zu diesem Aufgabenblatt auch die Powerpoint-Folien „Einführung in vvvv“. Sie erklären Schritt für Schritt (Tutorial-Stil) die Basisinformationen zu vvvv, Grundbegriffe, erste Programme usw. Den Umgang mit „vvvv“ erleichtern außerdem diese Ressourcen:**

- Getting Started (Grundlagen zu vvvv gamma mit Links zu Video-Tutorials): `https://thegraybook.vvvv.org/reference/getting-started/overview.html`
- Übersicht über die Tastaturbelegung: `https://thegraybook.vvvv.org/reference/hde/keyboard-shortcuts.html`

## 1) Das Arduino mit „vvvv“ sprechen lassen

Wie Sie in Blatt 3 gesehen haben, lässt sich das Arduino-Board „Standalone“ verwenden: Sie entwickeln die Logik in der Arduino IDE, kompilieren sie und spielen sie auf das Board. Danach arbeitet das Board als autarkes System: Es nimmt Werte von Sensoren entgegen, verarbeitet sie und gibt Feedback/Rückmeldung über Aktoren (LEDs, Motoren, Displays usw.) an die Außenwelt.

In den folgenden Aufgaben dient das Arduino-Board als „Interface“ (Schnittstelle). Die Werte der angeschlossenen Sensoren gehen direkt an den Computer (hier: an das Programm „*vvvv*“) und werden dort weiterverarbeitet. Umgekehrt kann das Board Daten von einem externen Programm empfangen (hier wieder „*vvvv*“; natürlich gehen auch andere Programme) und an Aktoren weitergeben. Das Arduino-Board hat dabei selbst keine Logik. Es ist nur die Schnittstelle zwischen der physischen Welt (Sensoren, Aktoren) und der Software auf dem Computer: **Die Logik entwickeln/programmieren Sie komplett in „*vvvv*“.**

**Für das erste Beispiel brauchen Sie folgende Komponenten:**

- PC mit installiertem „*vvvv gamma*”
- Arduino-Board (z. B. UNO), USB-Kabel
- Steckbrett („Breadboard”)
- Potentiometer/Drehregler, Größe 13mm, 1kOhm
- Externe LED, rot (2 Pins)

### 1.a) Wert eines Analogsensors (Potentiometer) in „vvvv“ darstellen

Sie übermitteln den Wert eines Potentiometers/Drehreglers über das Arduino-Board an „*vvvv*“ und stellen ihn dort in geeigneter Form dar. Bei dieser Art des „*Interfacings*“ legen Sie auf dem Arduino-Board zuerst die Konfiguration der Ein-/Ausgänge, die Art der Datenübertragung usw. fest. Erst dann können Sie in „*vvvv*“ auf die Daten zugreifen.

**Schritt 1:** Als Erstes entwickeln Sie ein einfaches Arduino-Programm, das sich um die Kommunikation kümmert. Übertragen Sie es per „*Copy & Paste*“ in die Arduino IDE (oder laden Sie die Datei aus *Moodle* bzw. dem *Datentausch*-Laufwerk „`P:`“), kompilieren Sie es und laden Sie es auf das Arduino-Board. Ergänzende Erklärungen:

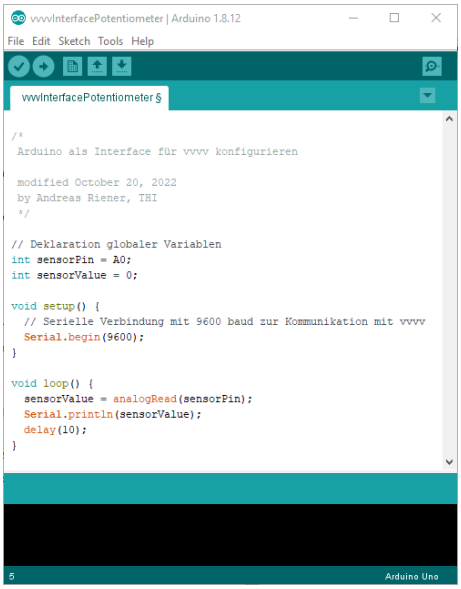
- Für die Kommunikation nutzen Sie die serielle Schnittstelle (die Sie in Blatt 3 schon zum Debugging genutzt haben): `Serial.begin(9600);` Die Baud-Rate (9600) gibt an, wie viele Symbole pro Sekunde übertragen werden; 9600 reicht in den meisten Fällen aus.
- Den Inhalt der Variablen `sensorValue` geben Sie wieder mit der Methode `Serial.println(sensorValue);` auf die serielle Schnittstelle aus.

```
/*
 Arduino als Interface für vvvv konfigurieren
 modified October 15, 2025
 by Andreas Riener, THI
 */

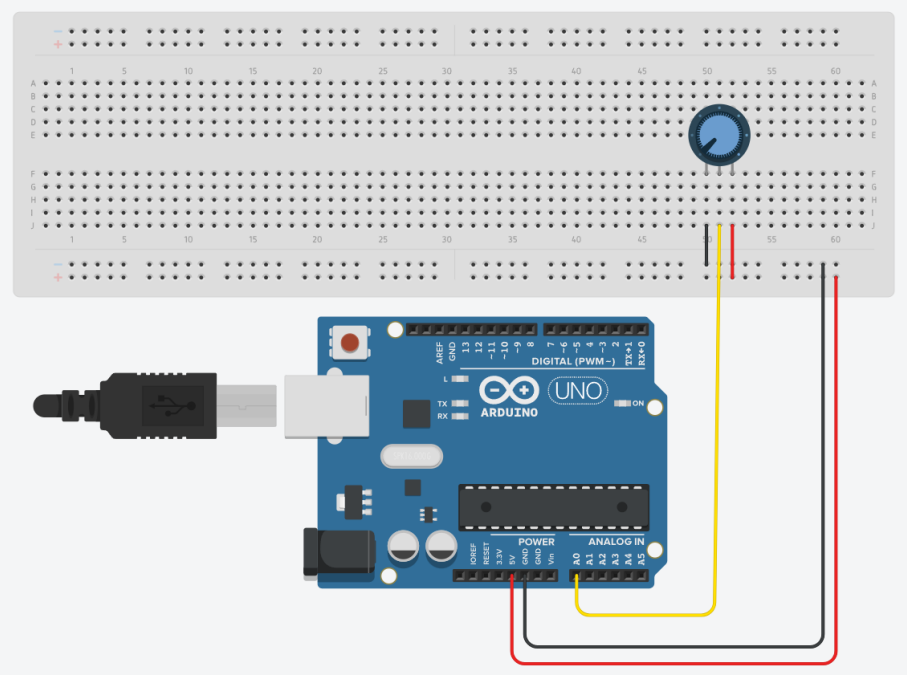
// Deklaration globaler Variablen
int sensorPin = A0;
int sensorValue = 0;
```

```
void setup() {
  // Serielle Verbindung mit 9600 baud zur Kommunikation mit vvvv
  Serial.begin(9600);
}

void loop() {
  sensorValue = analogRead(sensorPin);
  Serial.println(sensorValue);
  delay(10);
}
```



**Schritt 2:** Bauen Sie eine einfache Schaltung wie unten dargestellt auf. Wenn Sie den Regler drehen, ändert sich sein Widerstandswert. Der aktuelle Wert (er entspricht der Position des Drehreglers) wird am analogen Eingang „A0“ des Arduino-Boards eingelesen.




**Hinweis: Bei all unseren Breadboards sind die Leisten oben/unten (für 0V/GND und 5V) in der Mitte geteilt!**

**Schritt 3:** Als Nächstes starten Sie „*vvvv*“ und entwickeln die Logik, die Daten vom Arduino-Board empfängt. Zentrales Element im „*vvvv*“-Patch ist der „`SerialPort [IO]`“-Node. Er empfängt die Werte des Arduino-Boards auf der seriellen Schnittstelle. Damit die Kommunikation funktioniert, muss die sog. „Baud-Rate“, mit der das Arduino-Board sendet, mit der in „*vvvv*“ eingestellten übereinstimmen. Unten sehen Sie das „*vvvv*“-Programm, das Sie jetzt nachbauen/selbst implementieren.

**Vorgangsweise**

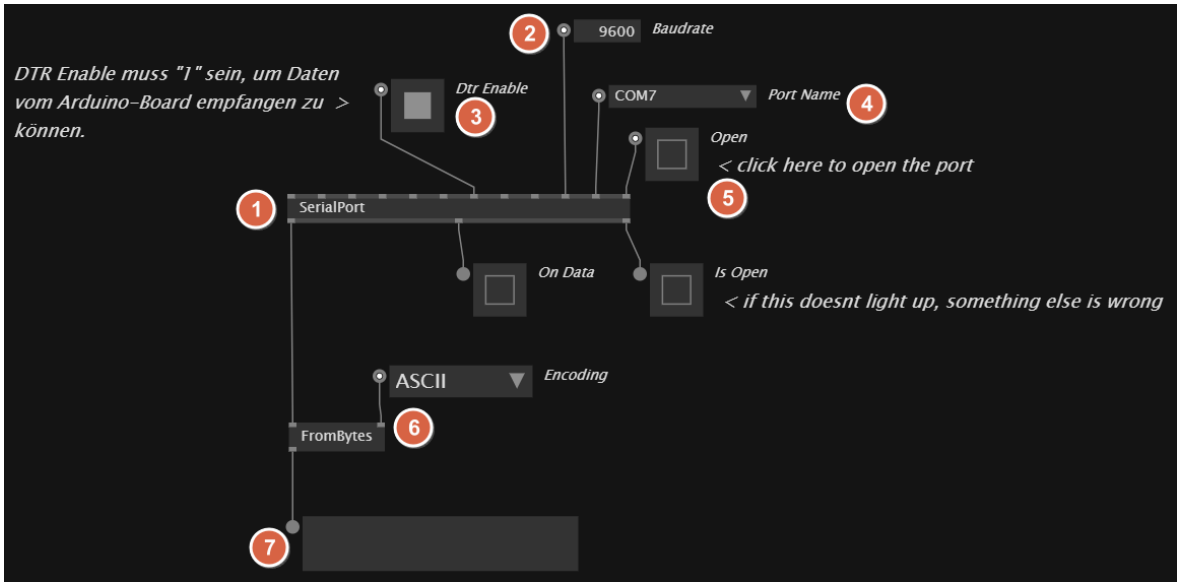
- Starten Sie „*vvvv*“ und legen Sie einen neuen, leeren Patch an
- Fügen Sie nach und nach die einzelnen Komponenten („*Nodes*“) wie folgt ein (**siehe Skizze unten!**):

**Hinweis: Die Nummern in Klammern entsprechen den Zahlen in den Kreisen der Patch-Screenshots**

(1) Ein Doppelklick (links) in das Fenster öffnet den „*NodeBrowser*“. Er listet alle verfügbaren „Nodes“ (Komponenten) auf; über das Eingabefeld können Sie filtern. Suchen Sie hier den „`SerialPort [IO]`“-Node und fügen Sie ihn in Ihren Patch ein. Vergrößern Sie den Node, damit Sie seine Optionen besser erkennen und leichter Verbindungen herstellen können.

- **Hinweis (Erinnerung):** Die einzelnen Nodes konfigurieren Sie am leichtesten im „**Inspector**“ (Quad-Menü mit der linken Maustaste öffnen, dort unter „Windows“ den „Inspector“ wählen, oder STRG + I).

– **Hinweis 2 (Erinnerung):** Für viele Nodes gibt es **Hilfe-Patches**. Ein Klick auf das Fragezeichen **im Inspector** zeigt die Hilfe-Patches an, in denen der betreffende Node verwendet wird; direkt **im Patch** öffnet **Node markieren + F1-Taste** den passenden Hilfe-Patch.




– Weitere Elemente („*Nodes*“) fügen Sie mit einem Doppelklick links in das „*vvvv*“-Programmfenster (Patch) ein. Konfiguration inkl. Datentypen der einzelnen Elemente:

(2) Starten Sie am Eingang „Baudrate“ einen Link und schließen Sie ihn mit der mittleren Maustaste ab. So erhalten Sie eine „`IOBox [Integer32]`“; geben Sie dort den Wert „`9600`“ ein.

(3) Starten Sie am Eingang „Dtr Enable“ einen Link, schließen Sie ihn mit der mittleren Maustaste ab und wählen Sie das Verhalten (Behavior) „`Toggle`“. Achten Sie darauf, dass „Dtr Enable“ aktiv ist.

(4) Starten Sie am Eingang „Port Name“ einen Link und schließen Sie ihn mit der mittleren Maustaste ab. So erhalten Sie einen Node vom Typ „`IOBox (Enumerations)`“, der sich korrekt mit den möglichen Optionen füllt. Mit der rechten Maustaste ändern Sie die Vorauswahl in der IOBox.

(5) Starten Sie auch am Eingang „Open“ einen Link. Schließen Sie ihn mit der mittleren Maustaste ab und wählen Sie Behavior „`Toggle`“.

(6) An den Ausgang des „*SerialPort*“-Nodes schließen Sie einen Node vom Typ „`FromBytes [Primitive.String]`“ an. Finden Sie diesen Node im NodeBrowser nicht, aktivieren Sie in den Filtereinstellungen des NodeBrowsers die „low level nodes“:



Erzeugen Sie an seinem Eingang „Encoding“ eine „`IOBox (Enumerations)`“ und stellen Sie dort „`ASCII`“ ein.

(7) Erzeugen Sie am Result-Ausgang des „*FromBytes*“-Nodes eine „`IOBox [String]`“. Klicken Sie mit der rechten Maustaste auf den Node: Configure → Font Size, z. B. „12“.

– Speichern Sie den Patch als „`Potentiometer an vvvv basic.vl`“ und starten Sie erst **DANACH** den Patch mit Rechtsklick auf (5) „Open“ (d. h., öffne die Verbindung zum Board). Im Ausgabefeld (7) sollten Sie jetzt den Wert des Potentiometers sehen.

**Frage 1:** Fällt Ihnen an der Ausgabe etwas auf? Was? Was könnte der Grund für dieses Verhalten sein (interpretieren Sie in eigenen Worten)?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!*

**Schritt 4:** Um das gerade gefundene Problem zu „beheben“, lesen Sie die Daten mit einer neueren Implementierung des Nodes ein: „`SerialPort4`“. SerialPort4 gehört zur VL.IO.Ports-Library in vvvv gamma und zur neueren, vollständig in .NET implementierten VL4-Architektur. (Dieser Knoten hat aktuell noch den Status „Experimental“, ist also noch nicht vollständig geprüft, und soll deshalb nicht in produktiven Systemen eingesetzt werden.) Öffnen Sie den Patch „`1b_SerialPort4_1.vl`“ und vergleichen Sie die Änderungen mit dem vorherigen Patch. Wandeln Sie die Ausgabe (Datentyp String) dann in einen numerischen Wert um, mit dem Sie später rechnen können (Tipps: Sehen Sie sich dazu bspw. den Node „`TryParse`“ (converts a String to a number) an).

**Frage 2:** Welche Änderungen stellen Sie im String-Ausgabefenster fest? Welche Ergänzung war nötig, um die Zeichenkette als Zahl auszugeben?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!*

## 2) Vereinfachte Interaktion mittels der „Firmata"

Wie Sie in Blatt 3 gesehen haben, lässt sich das Arduino-Board „Standalone" verwenden: Sie entwickeln die Logik in der Arduino IDE, kompilieren sie und spielen sie auf das Board. Danach arbeitet das Board autonom, also ohne Computer. Für interaktive Anwendungen eignet sich das natürlich nicht. In Aufgabe 1 (oben) haben Sie gesehen, dass Sie Sensorwerte auch über das Arduino-Board an „*vvvv*" übermitteln können. In der Standardversion (mit dem „`SerialPort [IO]`"-Node) ist das aber nicht besonders flexibel: Neue Projekte bzw. kleine Änderungen im vvvv-Programm erfordern meist auch eine angepasste Konfiguration des Arduino-Boards.

Mit dem sog. Firmata-Sketch (für das Arduino-Board) und einem speziellen „`FirmataDevice [IO.Firmata]`"-Node betreiben Sie ein Arduino-Board (in „*vvvv*") dagegen als flexibles und sehr mächtiges Interface. Wie das geht, sehen Sie in der nächsten Aufgabe.

### Was ist die Firmata eigentlich?

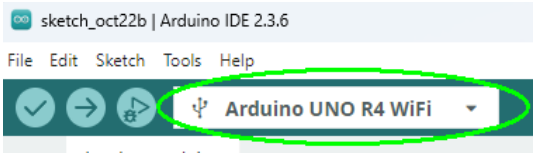
Die „*Firmata*" ist ein generisches Protokoll, über das Mikrocontroller mit beliebiger Software auf dem Computer kommunizieren. Ohne sie müssten Sie dafür eigene Programme auf dem Arduino-Board schreiben, die genau angeben, welche Pins angeschlossen sind und welche Funktionen sie nutzen. Außerdem müssten Sie sicherstellen, dass die Empfängerseite das Datenpaket richtig dekodiert, damit sie mit den empfangenen Werten arbeiten kann.

Mit der „*Firmata*" greifen Sie bequem auf Sensoren und Aktoren des Mikrocontrollers zu, ohne sich um Kommunikation und Codierung kümmern zu müssen.

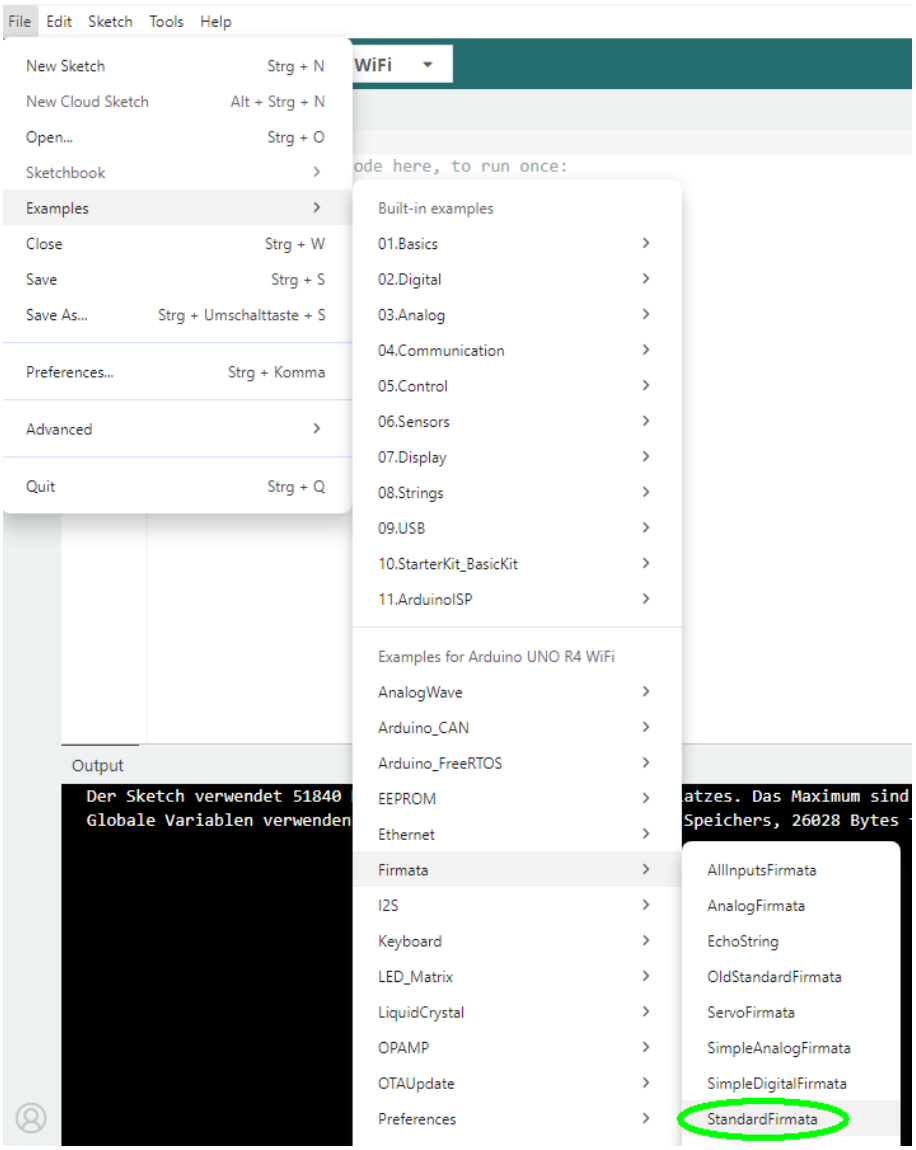
### Vorbereitung: Firmata auf das Arduino-Board laden

Der große Vorteil des „`FirmataDevice [IO.Firmata]`"-Nodes in „*vvvv*": Mit der „*Firmata*" müssen Sie sich nicht mehr um die Kommunikation zwischen Arduino-Board und „*vvvv*" kümmern.

Haben Sie in Ihrer Arduino IDE schon einen Sketch für das Arduino Uno R4 WiFi Board kompiliert bzw. das Board über den Board Manager der IDE hinzugefügt, sollte die Firmata-Library bereits installiert sein. Prüfen Sie das: Laden Sie bei aktiviertem Arduino Uno R4 Board




über File / Examples im Abschnitt „Examples for Arduino Uno R4 WiFi" den Eintrag Firmata / StandardFirmata.




Finden Sie in Ihrer IDE keinen Eintrag „Firmata“, wenden Sie sich bitte an den Betreuer bzw. installieren Sie die neueste Version der Firmata (by Firmata Developers) über den Library Manager der Arduino IDE (Menü „Tools“ – „LibraryManager“ – „Firmata“ ins Suchfeld eingeben – „Install“).

Kompilieren Sie zunächst den StandardFirmata-Sketch, der sich öffnet.

Leider enthält die momentane Version der Firmata-Library offensichtlich noch einen Fehler. Deshalb erhalten Sie wahrscheinlich diese Fehlermeldung:

```
#error "Please edit Boards.h with a hardware abstraction for this board"
```

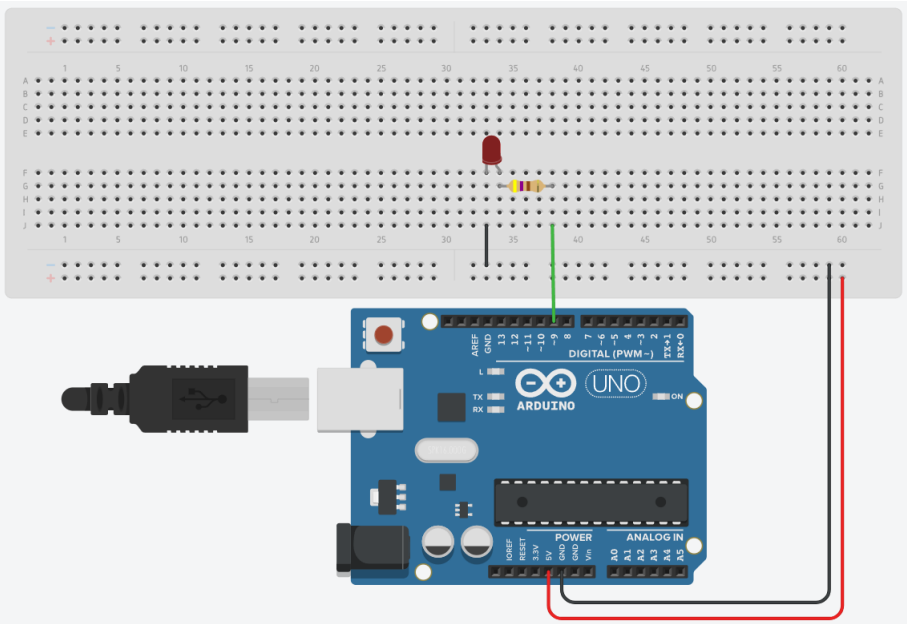
Tauschen Sie in diesem Fall die Datei `boards.h` in dem Ordner, den die Fehlermeldung angibt, gegen die Datei `boards.h` aus, die über den Datentausch „P:“ bzw. Moodle bereitsteht.

Jetzt sollte der Sketch fehlerfrei kompilieren, und ab sofort steuern Sie das Board bequem von „*vvvv*“ aus an, Sie „fernsteuern“ es quasi. **Schließen Sie bitte die Arduino IDE!**

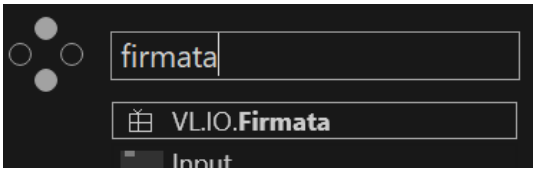
Im Folgenden geht es um einige Schaltungen, die Sie schon im Arduino-Aufgabenblatt (Blatt 3) gelöst haben, dort durch Programmieren des Arduino-Boards. **Jetzt implementieren Sie die Logik ausschließlich in „*vvvv*“. Am Arduino-Board selbst (bzw. im Arduino-Code) ändern Sie nichts mehr!**

### 2.a) Pulsweitenmoduliertes (PWM) Dimmen einer LED über „vvvv“

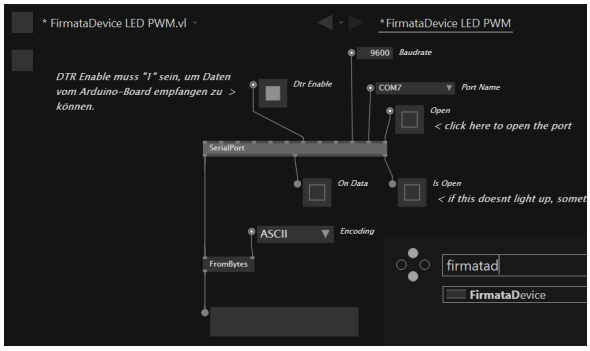
**Schritt 1:** Bauen Sie die einfache Schaltung unten (wieder) auf. Sie erinnern sich vielleicht: Die Schaltung (rote LED + Vorwiderstand 470 Ohm) ist identisch mit der aus Blatt 3.

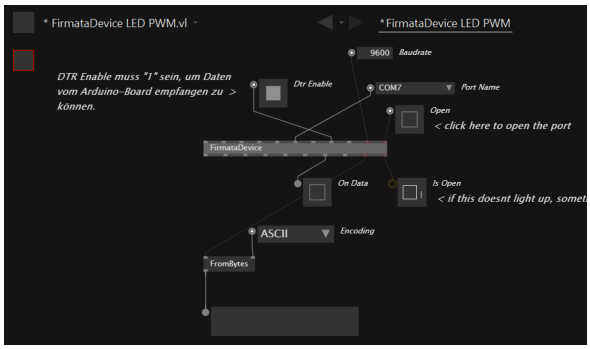



**Schritt 2:** Für die Logik kopieren Sie zuerst das Programm „`Potentiometer an vvvv basic.vl`“ aus Aufgabe 1, benennen es um in „`FirmataDevice LED PWM.vl`“ und öffnen es in „*vvvv*“. Ersetzen Sie den „*SerialPort*“-Node durch einen „*FirmataDevice*“-Node. Verwenden Sie die Firmata in diesem Patch zum ersten Mal, binden Sie zuerst die Library VL.IO.Firmata ein: mit einem Klick im NodeBrowser auf „VL.IO.Firmata“ mit dem Paketsymbol.



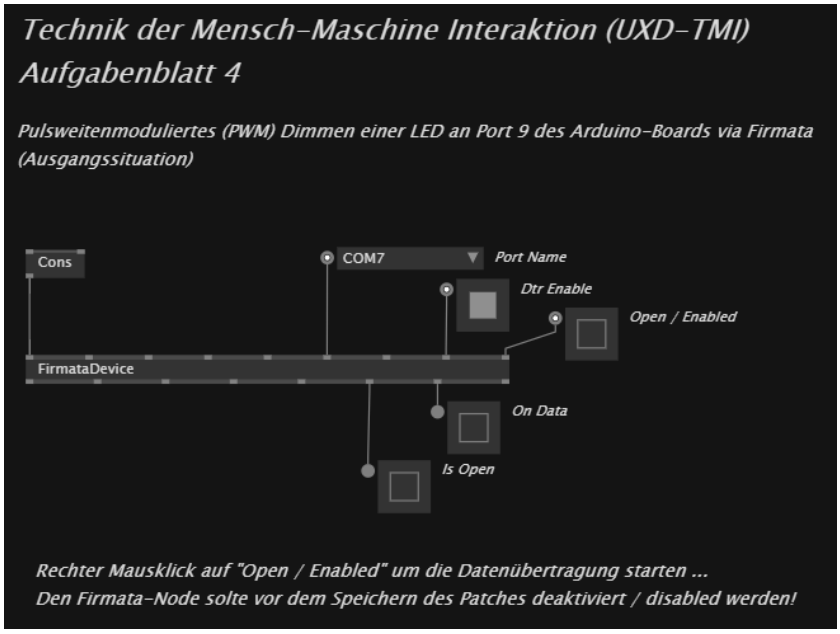

Danach: Doppelklick auf den „*SerialPort*“-Node, „Firmata“ ins Suchfenster eingeben (davor zweimal die „ESC“-Taste drücken, um im NodeBrowser 2 Hierarchieebenen aufzusteigen), „`FirmataDevice [IO.Firmata]`“ auswählen. Das Ergebnis sollte etwa so aussehen wie in der Abbildung unten rechts („FirmataDevice ersetzt SerialPort“).



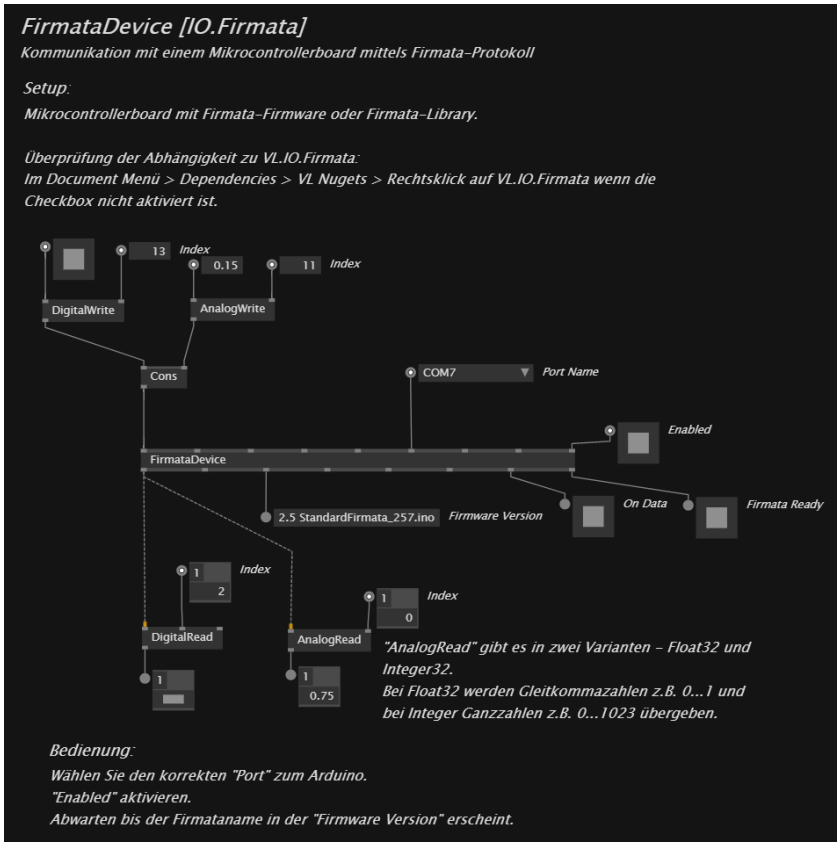




Passen Sie ggf. die Größe des „*FirmataDevice*“-Nodes an und prüfen Sie, ob alle Ein- und Ausgangspins korrekt angeschlossen sind (rote Pins sind fehlerhaft zugeordnet). Löschen Sie fehlerhafte Zuordnungen (z. B. eingangsseitig „Baud Rate“ und „Open“) und weisen Sie sie neu/korrekt zu (wenn Sie die Maus über die kleinen Quadrate am „*FirmataDevice*“-Node bewegen, sehen Sie, welche Funktion damit verknüpft ist). Damit es übersichtlicher wird, können Sie die einzelnen IOBoxen noch verschieben.

Mit dem „`Cons [Collections.Spread]`“-Node fassen Sie `AnalogWrite` und `DigitalWrite` des Arduino-Boards zusammen und verbinden das Ergebnis mit dem 1. Eingangspin des „`FirmataDevice [IO.Firmata]`“. **Das Ergebnis zeigt die folgende Abbildung**; Ihr Zwischenstand sollte ungefähr gleich aussehen ;-)




Jetzt definieren Sie die Pins für Ein-/Ausgabe, die Sie am Arduino-Board verwenden wollen, so wie Sie es auch in der Arduino IDE tun würden bzw. getan haben. Dafür gibt es „`AnalogWrite`“- und „`DigitalWrite`“-Nodes, die Sie mit dem „*Cons*“-Node konkatenieren und an den „*FirmataDevice*“-Node anschließen. Ist Ihnen die Funktion des „*Cons*“-Nodes unklar, sehen Sie sich seine Hilfe (F1) an. Zum „*FirmataDevice*“-Node gibt es in vvvv gamma leider noch keinen Hilfe-Patch. Ein Hilfe-Patch könnte aber etwa so aussehen:




Hier wird gleichzeitig Ausgabe-Pin 13 als Binärschalter (ein/aus) und Ausgabe-Pin 11 als Analogwert (0-1) verwendet.

**Frage 3:** Wie bringen Sie mit der Firmata eine LED an Pin 9 des Arduino-Boards zum Leuchten (einfaches Ein- und Ausschalten mit einer IOBox)? Beschreiben Sie es zuerst in Textform. Bauen Sie danach die Schaltung auf dem Breadboard auf und erweitern Sie den „*vvvv*“-Patch um die notwendigen Nodes (Abgabe Screenshot bzw. Foto des *.vl-Patches sowie den .vl-Patch als Datei).

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!*

**Frage 4:** Jetzt soll v4 zusätzlich eine weitere LED unabhängig von der ersten „dimmen“ (Helligkeitswert analog steuern). Wie müssen Sie Ihre Arduino-Schaltung ergänzen? Welchen Arduino-Pin verwenden Sie für die zweite LED (und warum)? Wie müssen Sie den „*vvvv*“-Patch verändern? (Abgabe Screenshot bzw. Foto des *.vl-Patches sowie den .vl-Patch als Datei).

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

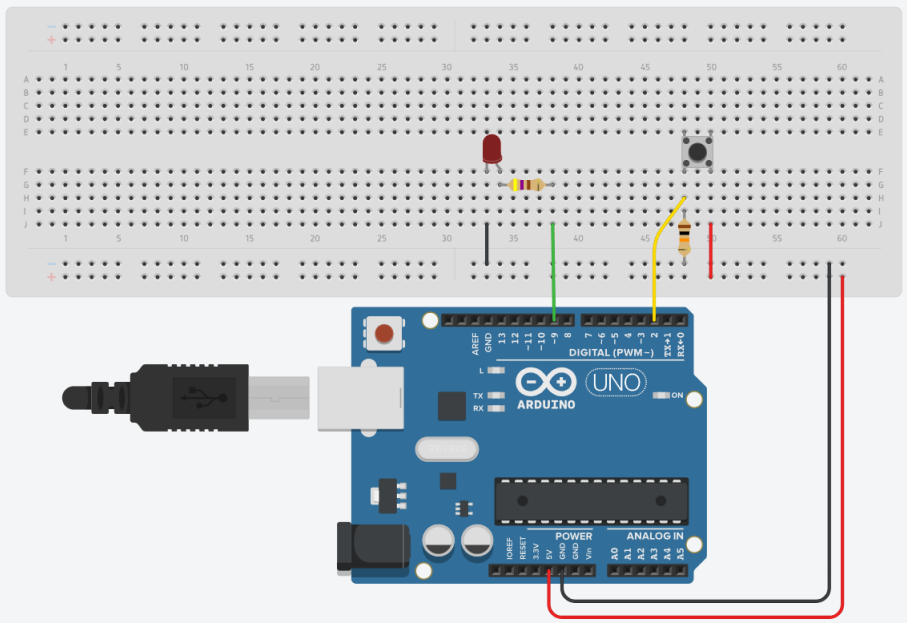
**Hinweis zu „vvvv“ und Arduino über Firmata**: Im Hintergrund tauschen „*vvvv*“ und das Arduino-Board über die serielle (USB-)Schnittstelle Konfigurationen aus. Deshalb **kann es bis zu 10 Sekunden dauern, bis ein „*vvvv*“-Patch tatsächlich mit dem Arduino-Board kommuniziert**. Planen Sie diese Wartezeit bei Ihren Tests ein (also: seien Sie nicht „zu“ ungeduldig…).

**ACHTUNG: Es darf immer nur ein Patch mit aktiviertem Firmata-Node offen/aktiv sein; sonst beeinflussen sich die Patches gegenseitig (der Patch im Vordergrund bekommt ggf. keine Daten).**

## 2.b) Verwenden eines „Buttons“ als Schalter für die LED

**Weiter geht’s mit Ein-/Ausgabe und Feedback: Der „*Pushbutton*“ soll jetzt die LED ein- bzw. ausschalten** (die „Standalone“-Version kennen Sie schon aus dem Arduino-Blatt 3).

**Schritt 1:** Ergänzen Sie die Schaltung am Breadboard um den „*Pushbutton*“ samt Verkabelung wie unten dargestellt (Pushbutton an Eingang 2, Pulldown-Widerstand 10kOhm, LED an Eingangs-Pin 9, Vorwiderstand 1kOhm).




**Schritt 2:** Ergänzen Sie jetzt selbst die Logik in „*vvvv*“ so, dass die LED binär ein-/ausgeschaltet wird: nicht mehr über den Slider, sondern über das Signal Ein/Aus, on/off, gedrückt/nicht gedrückt des Tasters an Pin 2 des „Arduino“-Boards. Speichern Sie den vorherigen Patch zuerst (d. h., bevor Sie etwas ändern) als „`Firmata LED PushButton.vl`“ ab. Für die Änderung müssen Sie noch Folgendes wissen bzw. berücksichtigen:

– Analog zu „`DigitalWrite`“ gibt es einen „`DigitalRead`“-Node, der Ihnen den Wert des jeweiligen Pins des Arduino-Boards liefert. Hängen Sie ihn an den „Pin States“-Pin des „*FirmataDevice*“-Nodes. Versuchen Sie zuerst, den Status des Schalters (on/off) in einer „IOBox“ anzuzeigen. Beachten Sie: Der Ausgang des „*DigitalRead*“-Nodes ist ein Spread. Um einen einzelnen Wert zu extrahieren, brauchen Sie einen „*GetSlice*“-Node.

– Für Feedback (d. h., ein Eingang des Arduino-Boards steuert das Verhalten an einem Ausgang des Arduino-Boards) müssen Sie in den Feedback-Kreis einen „`FrameDelay`“-Node einfügen, damit zeitliche Constraints eingehalten werden (**siehe Unterlagen zum Tutorial**).

**Frage 5:** Wie sieht Ihre Schaltung aus? Funktioniert sie wie gewünscht? Was macht der „*FrameDelay*“-Node genau, und was passiert, wenn Sie ihn nicht verwenden? Wo hatten Sie Probleme? (Abgabe Screenshot bzw. Foto des *.vl-Patches sowie den .vl-Patch als Datei).

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

### 2.c) Verwenden eines „Drehreglers“ zum Steuern der LED-Helligkeit

Im letzten Schritt regeln Sie die Helligkeit der LED mit einem Drehregler (Potentiometer). Speichern Sie den vorherigen Patch zuerst (d. h., bevor Sie etwas ändern) als „`Firmata LED Potentionmeter.vl`“ ab.

**Schritt 1:** Ändern Sie den Aufbau am Breadboard: Der Taster samt Kabel kann weg, der Drehregler kommt dazu. Schließen Sie ihn an Port 3 bzw. A3 des Arduino-Boards an (in vvvv lesen Sie ihn über einen „`AnalogRead`“-Node aus). Legen Sie dazu den linken und den rechten Pin des Drehreglers auf 0V (GND) und 5V. Der mittlere Pin liefert dann je nach Stellung des Reglers einen Wert von 0…5 Volt (in vvvv: 0…1 als Realzahl bzw. 0…1023 als Integer).

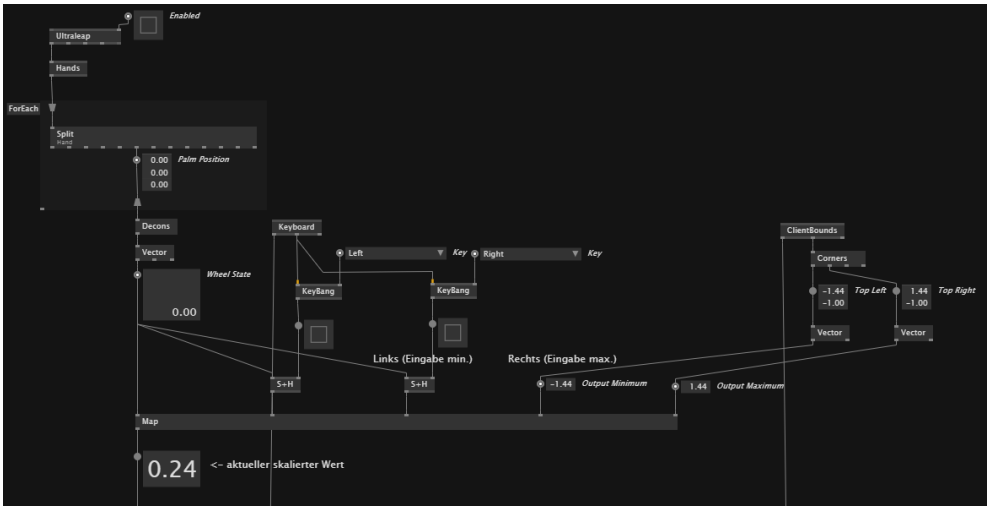
**Schritt 2:** Ändern/ergänzen Sie jetzt in „*vvvv*“ die Programmlogik so, dass der Wert (die Stellung) des Drehreglers die Helligkeit der LED kontinuierlich regelt (ähnlich wie in Aufgabe 2.a). Verwenden Sie einen „`AnalogRead`“-Node (Wert des Drehreglers an Eingabeindex 3) und versuchen Sie zuerst, den Wert des Drehreglers in einer IOBox anzuzeigen. Wenn das funktioniert, vervollständigen Sie die Schaltung, sodass dieser Wert die LED-Helligkeit regelt. Dazu verwenden Sie einen „`FrameDelay`“- und dahinter einen „`AnalogWrite`“-Node (Ausgabe an Index/Pin 9). Hinweis: Es gibt zwei verschiedene „*AnalogWrite*“-Nodes; sie erwarten als Input entweder 0...1 (Float32) oder 0...1023 (Integer32).

**Frage 6:** Wie sieht Ihre Schaltung aus? Funktioniert sie wie gewünscht? Wo hatten Sie Probleme? (Abgabe Screenshot bzw. Foto des *.vl-Patches sowie den .vl-Patch als Datei).

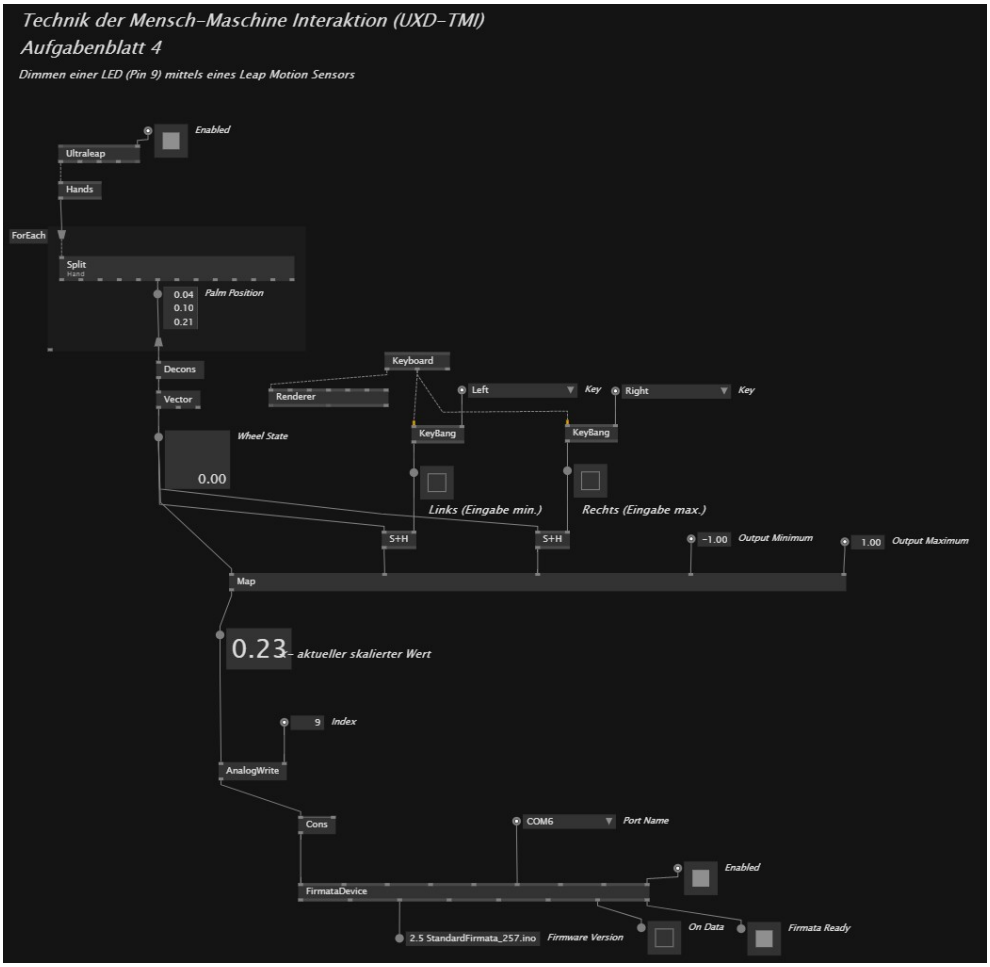
*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

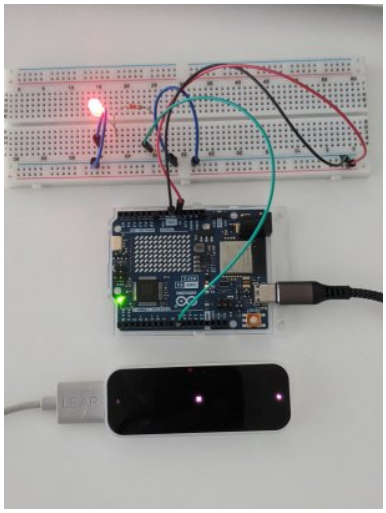
### [Variante „LeapMotion“ steuert die LED-Helligkeit]

…setzen wir im Praktikum nicht um. Am Dozenten-PC zeigen wir Ihnen aber, wie einfach sich statt der Eingabe über Arduino/Firmata andere Eingabegeräte für die Steuerung via Arduino einsetzen lassen:




Funktion „Leap Motion“ aus VA02 (Blatt 2)





## 3) Eine „sinnvolle Anwendung“: Temperaturanzeige

Aus der Temperatur lassen sich viele unterschiedliche Informationen ableiten. Mit einem Temperatursensor erfassen Sie diese Daten als Eingabesignal im Computer. Im letzten Beispiel dieses Aufgabenblatts verarbeitet vvvv den eingelesenen Wert eines Temperatursensors und sendet ihn an eine RGB-LED, die so die Temperatur eines Heißgetränks sichtbar macht.

### 3.a) Temperatursensor auslesen

**Schritt 1:** Ändern Sie die Schaltung auf dem Steckbrett („*Breadboard*“).

Zuerst muss vvvv das Signal des Temperatursensors empfangen. Dazu muss auf dem Arduino-Board wieder die StandardFirmata geladen sein. In diesem Aufbau verwenden Sie einen Temperatursensor vom Typ LM35 (erinnern Sie sich an Vorführaufgabe 2: Dort hatten wir den LM35 in einer Heizungssteuerung eingesetzt. Das ist derselbe Sensor.)

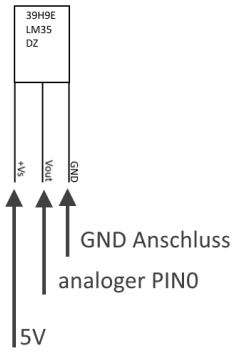
Die Belegung finden Sie im Datenblatt (`http://www.ti.com/lit/ds/symlink/lm35.pdf`) oder hier:

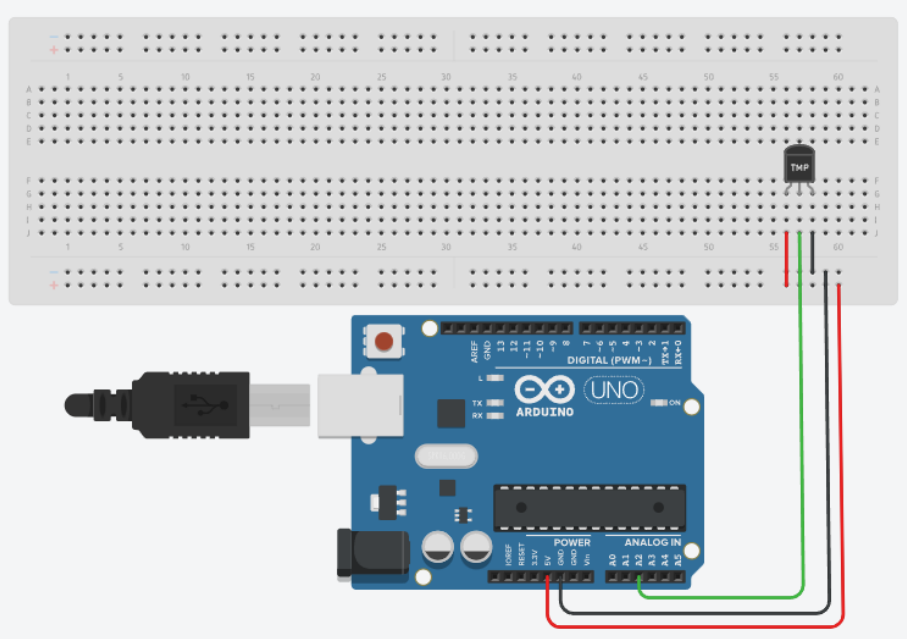
- Temperaturbereich: -55 bis 150° Celsius (mit Pull-down) bzw. **+2 bis 150° Celsius (ohne Pull-down Widerstand)**
- Der LM 35 liefert 0 V bei + 0°C
- Empfindlichkeit: 10 mV/° Celsius
- Anschlussschema siehe nachfolgend

**Anschlussschema:**

- Den rechten Pin des Sensors mit dem 5V-Anschluss des Arduino-Boards verbinden
- Den mittleren Pin mit dem „Analog-In“-Port A0
- Den linken Pin mit „Ground“ (0V)

So zeigt es auch die Schaltungsskizze unten.






**Temperaturcharakteristik** am Arduino-Board (mit 5V Versorgungsspannung) und in Standardbeschaltung ohne Pull-down Widerstand:

- 10mV (=0,01V) pro °C → damit bei +150°C: 0,01V * 150 = 1,5 V; 1,5 V / 5V = 0,3; der max. Ausgangswert des LM35 bei Versorgungsspannung von 5V ist somit für 150°C: **0,3**.
- **ACHTUNG: Liefert der LM35 einen unplausiblen Rohdatenwert größer als 0,3, ist er vermutlich falsch angeschlossen (+5V und GND vertauscht). Bitte das Board sofort ausschalten/abstecken und +5V und GND tauschen. Der LM35 wird sehr heiß – verbrennen Sie sich nicht!**
- Die Umrechnung in einen konkreten Temperaturwert in °C erklären wir weiter unten.

Öffnen Sie den Patch „`FirmataDevice TempSens.vl`“ aus dem Datenaustausch („P:“) und erweitern Sie ihn um folgende Komponenten (siehe Screenshot weiter unten):

**(1) Temperatursensor-Wert auslesen**

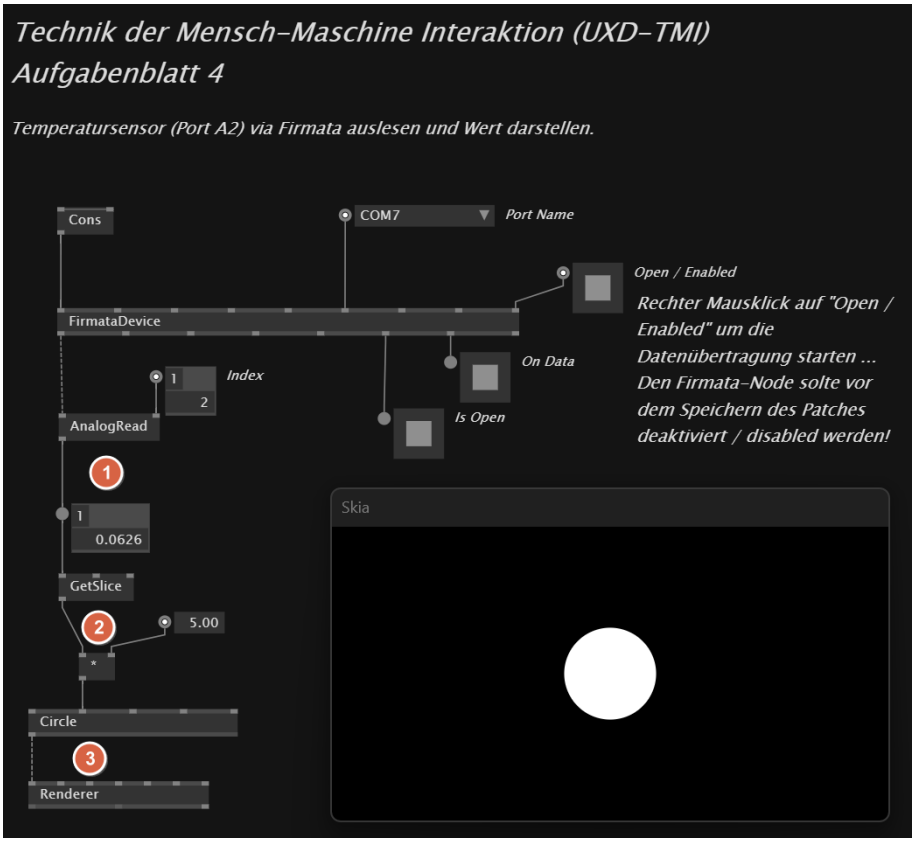
Weil der Temperatursensor an „Pin A0“ angeschlossen ist, muss am „`AnalogRead`“-Node der Index 0 eingestellt sein. Ist Ihre Schaltung korrekt, kommt am Ausgangspin ein Wert zwischen 0 und 1 (Float) an.

**(2) Größenkorrektur**

Je nach aktueller Temperatur ist der Kreis im Renderer womöglich sehr klein. Mit einem „`* (Value)`“-Node und einem Multiplikationsfaktor von z. B. „5“ stellen Sie die gewünschte Größe ein bzw. korrigieren sie.

**(3)** Damit die Größe eines Kreises den Messwert zeigt, fügen Sie einen „`Circle [Graphics.Skia.Layers]`“-Node ein. Das Ergebnis zeigt anschließend ein „`Renderer [Graphics.Skia]`“-Node an.

Speichern Sie Ihren Patch als „`FirmataDevice TempSens v2.vl`“ ab.




**(4) Korrekte Temperatur ausgeben**

Der Ausgang des Temperatursensors zeigt bisher einen schwer interpretierbaren Wert (ca. 0,0626), der keiner konkreten Temperatur entspricht.

Für die **Berechnung/Ermittlung der Temperatur in °C** können Sie folgenden Ausschnitt eines Patches verwenden (dieser vvvv-Programmteil steht auch als „`LM35Temperature.vl`“ in Moodle bzw. im Datentausch bereit; Sie können ihn per Copy & Paste einfügen):




**Erklärung:** Die LM35-Rohdaten (Gleitkomma, 0,0 bis 1,0) multiplizieren Sie mit 5V und erhalten einen Spannungswert; diesen dividieren Sie durch 0,01 Volt (=10 mV) und erhalten den Temperaturwert. Weil der Sensor ungenau ist, korrigieren Sie die Temperatur mit einem zusätzlichen Skalierungsfaktor auf den Wert, den Sie tatsächlich mit einem Thermometer messen.

Vervollständigen Sie die Schaltung, geben Sie den konkreten Temperaturwert in einer IOBox aus und steuern Sie mit dem Temperaturwert wieder die Anzeige des Kreissegments…

Speichern Sie Ihren Patch als „`FirmataDevice TempSens final.vl`“ ab.

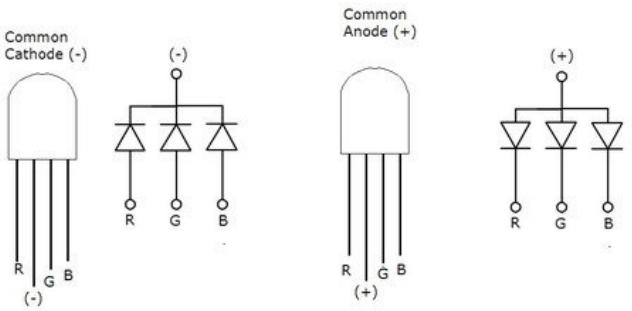
**Frage 7:** Welchen Wert hätte der Skalierungsfaktor, wenn der LM35-Sensor perfekt/korrekt funktioniert? Was ist Ihnen sonst beim Einsatz des LM35 in Ihrem ersten vvvv-Programm aufgefallen?

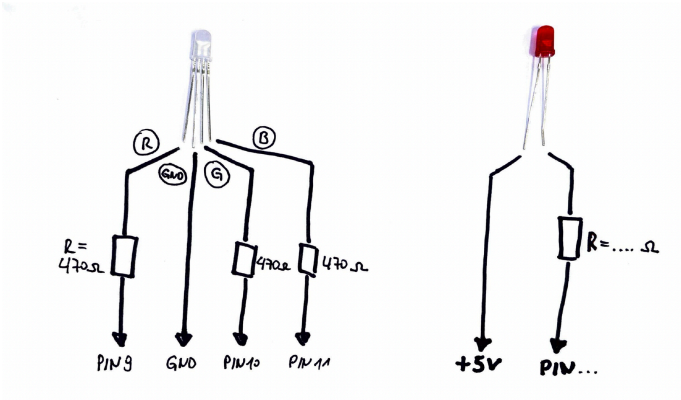
*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

### 3.b) RGB-LED mit Temperatursensor steuern

**Beachten Sie folgende Punkte:**

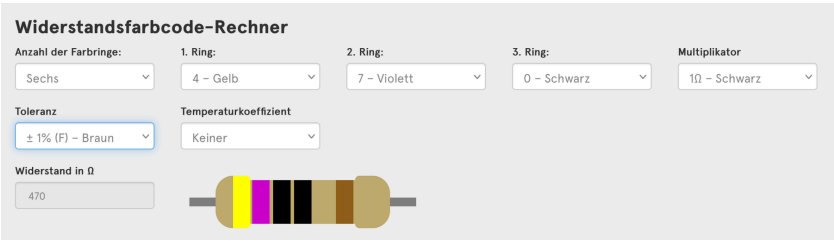
– Die RGB-Leuchtdiode, die wir verwenden, hat insgesamt 4 Pins: je einen Pin für die 3 Farbkanäle (rot, grün und blau) und einen gemeinsamen Pin entweder für die Versorgungsspannung +5 V oder für 0 V/GND. Eine LED mit gemeinsamer Versorgungsspannung heißt „*Common Anode*“ (gemeinsame Anode), eine mit gemeinsamem GND (0V) „*Common Cathode*“ (gemeinsame Kathode oder common ground), siehe Skizzen unten. Das bedeutet: Die 3 kleinen Einzelleuchtdioden in der LED teilen sich den Pin für die Versorgungsspannung oder für GND. Der „Plus-Pin“ (Anode) ist bei Common Anode der längste Pin der LED, der „Minus- bzw. 0V-Pin“ bei Common Cathode. Die Pins für R und G sind kürzer (beide etwa gleich lang), und der Pin für Blau ist am kürzesten (auch das zeigt die Skizze).



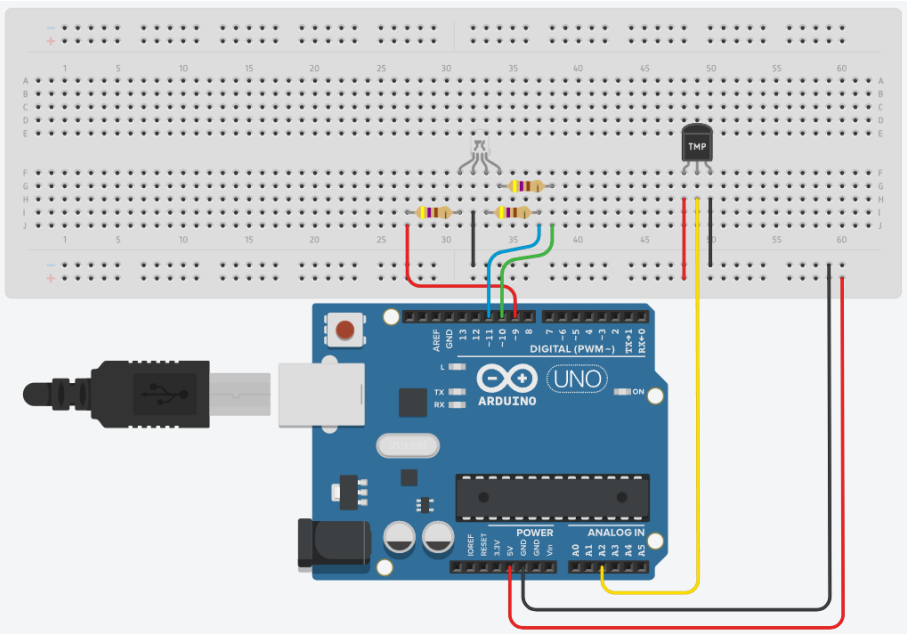

Wir haben beide Typen im Labor; für VA04 (dieses Blatt) verwenden wir die RGB-LED mit gemeinsamer Kathode (Common Cathode). Quellen: `https://moderndevice.com/wp-content/uploads/2013/09/rgb_large.jpg` (Typen) und Prof. Dr. Andreas Riener (Skizze).

- Vorwiderstände ermitteln: Die Durchlass-/Sperreigenschaften der einzelnen Leuchtdioden (R, G, B) (hier für Common Cathode, gemeinsamer GND) sind unterschiedlich. Deshalb brauchen Sie auch unterschiedliche Vorwiderstände (Berechnung z. B. auf `https://de.farnell.com/widerstand-farbcode-rechner`):
  - **Rot**: Durchlassspannung **1.7 V**, Betriebsspannung 5 V, Betriebsstrom 8 mA; (5 V-1.7 V)/ 8mA = 412 Ω → nächsthöherer verfügbarer Wert **= 470 Ω** (Gelb, Violett, Schwarz, Schwarz, Braun)
  - **Grün**: Durchlassspannung **2.2 V**, ergibt 350 Ω → nächsthöherer verfügbarer Wert **= 470 Ω** (Gelb, Violett, Schwarz, Schwarz, Braun)
  - **Blau**: Durchlassspannung **2.4 V**, ergibt 325 Ω → nächsthöherer verfügbarer Wert **= 470 Ω** (Gelb, Violett, Schwarz, Schwarz, Braun)



**Schritt 2:** Arduino-Board für die Kommunikation mit „*vvvv*“ vorbereiten. Die Standard-Firmata (Datei „`StandardFirmata_257.ino`“) ist schon auf dem Arduino-Board, dieser Schritt entfällt also – Juhu!

**Schritt 3:** Bauen Sie Ihre Schaltung nach dem Schaltplan unten (Tinkercad-Skizze) auf Ihrem Breadboard nach.




Gemeinsamer Pin auf GND (common cathode)

**Schritt 4:** Steuern Sie als Nächstes die rote, grüne und blaue LED in der RGB-LED über den Wert des Temperatursensors an und bringen Sie sie zum Leuchten (über Pins 9, 10, 11).

**Frage 8:** Wie sieht Ihre Schaltung jetzt aus? Wie verhält sie sich? Können Sie den Wert des Temperatursensors beeinflussen? (Wie?) (Abgabe Screenshot bzw. Foto des *.vl-Patches sowie den .vl-Patch als Datei).

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

**Schritt 5:** Jetzt bauen Sie noch die Steuerung für die LED in das v4-Programm ein. Erweitern Sie dazu den Patch aus Aufgabe 3.a. Speichern Sie den Patch unter „`FirmataBoard TempSens RGB.vl`“ ab.

(1) Rechnen Sie den Output des Sensors auf die Hue-Werte eines gewählten Farbspektrums um. Fügen Sie dafür den Node „`Map [Math.Ranges]`“ ein und verbinden Sie den Output des Nodes, der den Temperaturwert darstellt, mit dem Input-Pin.

(2) Geben Sie einen minimalen und einen maximalen Wert für den Sensor-Output an und dazu einen minimalen und einen maximalen Zielwert. Dafür fügen Sie jeweils eine „IOBox“ ein.

Die Zielwerte bestimmen die Farbe der RGB-LED bei der minimalen und der maximalen gemessenen Temperatur des Sensors. Wenn Sie mit der mittleren Maustaste auf die IOBoxen „Output Minimum“ und „Output Maximum“ klicken, blenden Sie mit „Show Slider“ einen Slider ein, mit dem Sie Werte festlegen.

(3) Den Wert, den der Temperatursensor aufgenommen hat, wandeln Sie jetzt in einen tatsächlichen Farbwert um. Dafür können Sie den „`FromHSV [Color.RGBA]`“-Node benutzen. Mit einer „IOBox“ zeigen Sie ihn auch im Patch an.

(4) Der „`RGBA (Split) [Color.RGBA]`“-Node ermittelt die Werte für den roten, grünen und blauen Kanal der LED.

(5) Jetzt verbinden Sie über „`FrameDelay`“-Nodes die einzelnen Farben mit dem Input der „`AnalogWrite`“-Nodes.

Speichern Sie Ihren Patch als „`FirmataBoard TempSens RGB.vl`“ ab.

**Frage 9:** Warum müssen die RGB LEDs an Pins mit PWM angeschlossen sein?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

**Schritt 6:** Zuletzt testen Sie, ob die Kopplung zwischen vvvv und Arduino funktioniert und ob Sie eine Änderung der Temperatur messen bzw. darstellen können. Dazu sprühen wir Kältespray auf den Sensor und beobachten, was passiert…

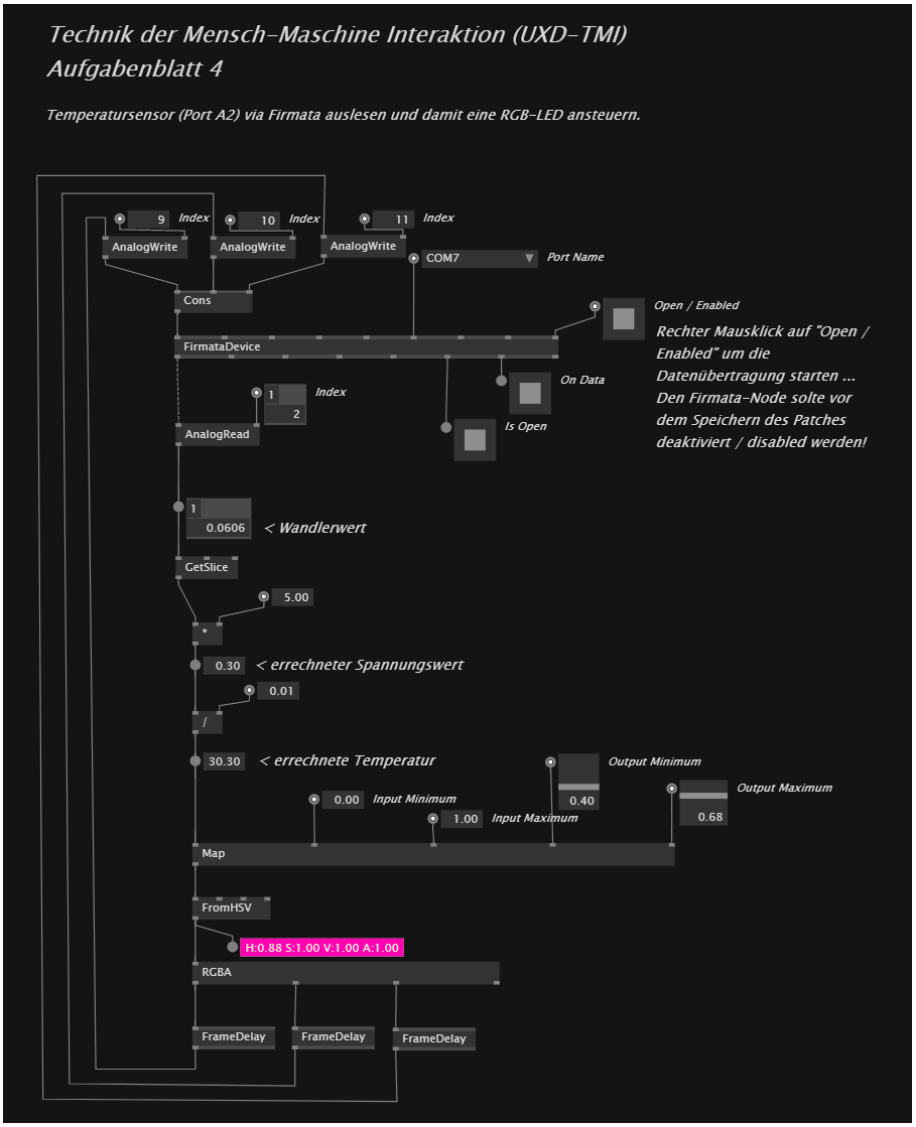
**Frage 10:** Was beobachten Sie? Welches Problem stellen Sie fest? Beschreiben Sie die Veränderung Ihrer LED.

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

**Frage 11:** Wodurch lässt sich dieses Problem beheben und Ihre Temperaturanzeige noch verbessern?

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Online-Protokoll (Moodle).*

**Beispiel**, wie die finale Schaltung aussehen könnte:




**BITTE BEACHTEN: Am Ende der Praktikumsstunde…**

- …bauen Sie die Schaltungen wieder ab
- …verpacken Sie sämtliche Bauteile wieder sauber und vollständig in der Schachtel: Widerstände zurück in die richtigen Klebestreifen, Drehregler/LEDs in die Säckchen usw.
- …verlassen Sie die Arbeitsplätze sauber und gereinigt.